ExpositoTOP — Portal técnico

RTVRPTW, Recomendador Turístico con Ventanas de Tiempo (TOPTW + GRASP). Documentación Docs-as-Code.

Documentación técnica generada con MkDocs + Doxygen

01/11/2026

Contenido

1. Inicio
2. Guía de usuario
3. Formato de las instancias
4. Preguntas frecuentes
5. Arquitectura
6. Algoritmo GRASP
7. Maven y su ciclo de vida
8. Órdenes y uso desde el IDE
9. Plugins configurados
10. Análisis con SonarQube
11. Incidencias y soluciones
12. Pruebas
13. Doxygen y MkDocs
14. Decisiones (ADR)
15. 0001 Migración a Maven
16. 0002 Ventana temporal desde los últimos campos
17. 0003 Aleatoriedad inyectable
18. 0004 Salida desacoplada de la consola
19. 0005 Estrategia de selección como enum
20. 0006 Docs-as-Code con Doxygen + MkDocs
21. Referencia API
22. Clases
23. Jerarquía
24. Miembros
25. Paquetes
26. Ficheros
27. Cambios

ExpositoTOP — RTVRPTW

Recomendador Turístico con Ventanas de Tiempo. Dado un conjunto de puntos de interés (POI) con su interés turístico, su horario y la duración de la visita, el sistema propone un itinerario de varios días que **maximiza el interés total** sin salirse de los horarios ni de la duración máxima de cada jornada.

Formalmente es el *Team Orienteering Problem with Time Windows* (**TOPTW**), un problema NP-difícil que el proyecto resuelve con la metaheurística **GRASP** (*Greedy Randomized Adaptive Search Procedure*).

Empezar

Compilar, ejecutar e interpretar los resultados.

Guía de usuario

Diseño

Clases, estructuras de datos y funcionamiento del GRASP.

Arquitectura · Algoritmo

Construcción con Maven

Ciclo de vida, órdenes y plugins configurados en el `pom.xml`.

Maven

Calidad

Análisis con SonarQube, incidencias detectadas y correcciones aplicadas.

SonarQube · Incidencias

Referencia de la API

Generada por Doxygen a partir de los comentarios del código.

Referencia API

Decisiones

Registro de decisiones de arquitectura (ADR).

ADR

Resumen rápido

```
mvn package                      # compila, ejecuta los tests y genera target/ExpositoTOP.jar
java -jar target/ExpositoTOP.jar --iterations 100 --seed 42 c101.txt
```

Estado del proyecto

Aspecto	Antes	Después
Construcción	NetBeans / Ant (`build.xml`)	Maven (`pom.xml`) + GitHub Actions
Tests automáticos	Ninguno	43 tests JUnit (lectura, modelo, GRASP, utilidades, CLI)
Documentación	Javadoc vacío en parte de las clases	Comentarios Doxygen (`\brief` + detalle) en todas las clases y métodos; portal MkDocs
Fallos funcionales conocidos	Lectura incorrecta de ventanas temporales en instancias `pr*`	Corregido (ver ADR 0002)

Sobre esta documentación

Las guías, la arquitectura y los ADR de este portal se redactaron con ayuda de un modelo de lenguaje (LLM) a partir del código fuente, y después se revisaron contra el propio código y los tests (actividad 9.3 de la práctica). La referencia de la API se genera automáticamente con Doxygen y MkDoxy en cada construcción.

Guía de usuario

Requisitos

Herramienta	Versión	Para qué
JDK	17 o superior	Compilar y ejecutar
Maven	3.6.3 o superior	Construir el proyecto
Doxygen + Graphviz	1.9+ / 2.40+	*(Opcional)* referencia del código y grafos
Python + MkDocs	3.9+	*(Opcional)* portal de documentación

Comprobación rápida:

```
java -version
mvn -v
```

Construir

```
git clone https://github.com/Andreaavr/ExpositoTOP.git
cd ExpositoTOP
mvn package
```

Se generan en `target/`:

- `ExpositoTOP.jar`: aplicación ejecutable.
- `ExpositoTOP-sources.jar` y `ExpositoTOP-javadoc.jar`: código fuente y documentación.
- `surefire-reports/`: resultados de los tests.

Ejecutar

JAR

```
java -jar target/ExpositoTOP.jar [--dir DIR] [--iterations N] [--seed S] [instancia ...]
```

Maven

```
mvn compile exec:java                                    # argumentos por defecto del pom
mvn compile exec:java -Dexec.args="--iterations 500 r101.txt"
```

Opción	Por defecto	Descripción
`--dir DIR`	`Instances/TOPTW`	Directorio donde están las instancias
`--iterations N`	`10000`	Construcciones GRASP por cada tamaño de RCL (3, 5 y 7)
`--seed S`	aleatoria	Semilla para obtener resultados reproducibles
`instancia ...`	las 29 de Solomon (`c1*`, `r1*`, `rc1*`)	Ficheros a resolver

> **Duración del experimento completo**
>
> Sin argumentos se reproduce el experimento original: 29 instancias × 3 tamaños de RCL × 10 000 iteraciones. Escribe el informe de cada solución, por lo que tarda bastante y genera mucha salida. Para pruebas, use `--iterations 100` y redirija la salida a un fichero (`> resultados.txt`).

Interpretar la salida

Por cada solución construida se imprime un bloque como este:

```
SOLUTION:
0 - 5 - 3 - 7 - 83 - 82 - 61 - 26 - 51 - 21 - 49 - 47 - 0
101 - 43 - 42 - 41 - 27 - 46 - 45 - 48 - 23 - 66 - 75 - 101
102 - 67 - 87 - 86 - 94 - 92 - 85 - 6 - 89 - 91 - 102
...
109 - 81 - 54 - 53 - 56 - 72 - 77 - 79 - 80 - 109
FEASIBLE SOLUTION: true
SCORE: 1470.0
TIME COST: 10043.516672012478
```

(Primera solución de `c101.txt` con `--seed 42`.)

- Cada línea de `SOLUTION` es una ruta (un día). Empieza y termina en su depósito: `0` para la primera ruta y `101`, `102`, … (número de POI + 1, + 2, …) para las siguientes. Todos los depósitos representan el mismo lugar físico (el hotel).
- `FEASIBLE SOLUTION` indica si se respetan todas las ventanas temporales y la duración máxima.

- `SCORE` es la suma del interés de los POI visitados (valor a maximizar).
- `TIME COST` es la suma de las horas de regreso al depósito de todas las rutas.

A continuación aparece una tabla por ruta con las coordenadas, la ventana `[READY TIME, DUE DATE]`, la hora de llegada y salida y la duración de la visita de cada nodo.

Tras las iteraciones de cada tamaño de RCL:

```
--> MEDIA: 1448.6
--> MEJOR SOLUCION: 1550.0
```

Resultados reproducibles

Con `--seed`, dos ejecuciones con los mismos argumentos producen exactamente la misma salida. Es lo que usan los tests de regresión para comprobar que las refactorizaciones no cambian el comportamiento del algoritmo.

Usar la biblioteca desde código

```
TOPTW problem = TOPTWReader.readProblem("Instances/TOPTW/c101.txt");
TOPTWGRASP grasp = new TOPTWGRASP(new TOPTWSolution(problem), new Random(42), null);
grasp.setStrategy(TOPTWGRASP.SelectionStrategy.FUZZY_ALPHA_CUT);
grasp.setAlpha(0.8);
double best = grasp.grasp(1000, 5);          // mejor puntuación
double mean = grasp.getLastAverageFitness(); // puntuación media
```

Generar la documentación

```
pip install -r requirements-docs.txt
doxygen Doxyfile                    # target/doxygen/html, latex y xml
make -C target/doxygen/latex        # target/doxygen/latex/refman.pdf
mkdocs serve                        # portal en http://127.0.0.1:8000
mkdocs build                        # portal estático en site/
python scripts/build_pdf.py         # docs/pdf/ExpositoTOP-documentacion.pdf
```

O todo a la vez con Maven: `mvn -Pdocs package`.

Formato de las instancias

Las instancias están en `Instances/TOPTW/` y siguen el formato de Cordeau (instancias `pr*`) adaptado por Righini y Salani al TOPTW, también usado para las de Solomon (`c*`, `r*`, `rc*`).

```
4 10 100 1                                  ← k v N t
0 200                                       ← D Q
0 40.00 50.00 0.00 0.00 0 0 0 1236          ← depósito (nodo 0)
1 45.00 68.00 90.00 10.00 1 1 1 912 967     ← POI 1
...
```

Primera línea: `k v N t`

Campo	Significado	Uso
`k`	—	No se usa
`v`	Número de rutas (vehículos / días)	`TOPTW.getVehicles()`
`N`	Número de POI	`TOPTW.getPOIs()`
`t`	—	No se usa

Segunda línea: `D Q`

No se usa.

Líneas de nodo: `i x y d S f a lista O C`

Campo	Significado	Uso
`i`	Número del nodo (0 = depósito)	—
`x`, `y`	Coordenadas	Matriz de distancias euclídeas
`d`	Duración de la visita	`getServiceTime(i)`
`S`	Puntuación (interés)	`getScore(i)`
`f`, `a`	—	`a` es la longitud de `lista`
`lista`	`a` valores	No se usa
`O`	Apertura de la ventana temporal	`getReadyTime(i)`
`C`	Cierre de la ventana temporal	`getDueTime(i)`

La lista tiene longitud variable

Como `lista` ocupa `a` campos, `O` y `C` **no están en una columna fija**: son siempre los **dos últimos** campos de la línea. La versión original leía las columnas 8 y 9, lo que solo es correcto cuando `a = 1` (como en las instancias de Solomon). En las instancias `pr*`, donde `a` vale 1, 2 o 4, se leían como ventana temporal valores de la lista. Ver ADR 0002.

Observaciones

- El nodo 0 es a la vez el punto de salida y de llegada de todas las rutas.
- La duración máxima de cada ruta es el cierre de la ventana del depósito (`C` del nodo 0).
- El tiempo de viaje entre dos nodos es su distancia euclídea.
- El fichero `Instances/TOPTW/formatInstances.txt` contiene la descripción original del formato.

Preguntas frecuentes

¿Por qué dos ejecuciones dan resultados distintos?

El GRASP es aleatorio. Use `--seed` para fijar la semilla y obtener siempre la misma salida.

¿Qué significan los depósitos 101, 102…?

Son copias virtuales del depósito 0 (el hotel). Cada ruta adicional necesita su propio nodo de inicio y fin en la lista doblemente enlazada, así que se numeran a partir de POI + 1. Al consultar coordenadas, horarios o distancias se tratan como el nodo 0.

¿Por qué algunos POI no se visitan nunca?

Si no pueden incluirse en ninguna ruta sin incumplir su ventana temporal o la duración máxima, el algoritmo no los inserta. En el TOPTW no es obligatorio visitar todos los POI.

Obtengo «Formato de instancia no válido»

El fichero tiene menos líneas de nodo que las indicadas en la primera línea o algún campo no es numérico. Revise el formato.

¿Cambian los resultados respecto a la versión original?

En las instancias de Solomon (`c*`, `r*`, `rc*`), no: con la misma semilla las rutas son idénticas (lo comprueba un test de regresión). En las `pr*` sí, porque se corrigió la lectura de las ventanas temporales.

¿Cómo cambio la estrategia de selección de la RCL?

Desde código, con `setStrategy(SelectionStrategy.RANDOM | FUZZY_BEST | FUZZY_ALPHA_CUT)` y `setAlpha(...)`. Por defecto se usa `FUZZY_ALPHA_CUT` con α = 0,8, igual que el original.

Arquitectura

Visión general

El proyecto es una aplicación Java de consola sin dependencias externas, organizada en tres paquetes:

Paquete	Contenido
`top`	Dominio del problema y algoritmo: modelo, lector, solución, evaluador, GRASP y programa principal
`es.ull.esit.utilities`	Utilidades genéricas: formato de tablas, ficheros, conjunto potencia, Bellman-Ford
`es.ull.esit.utils`	Tipos auxiliares (`Pair`)

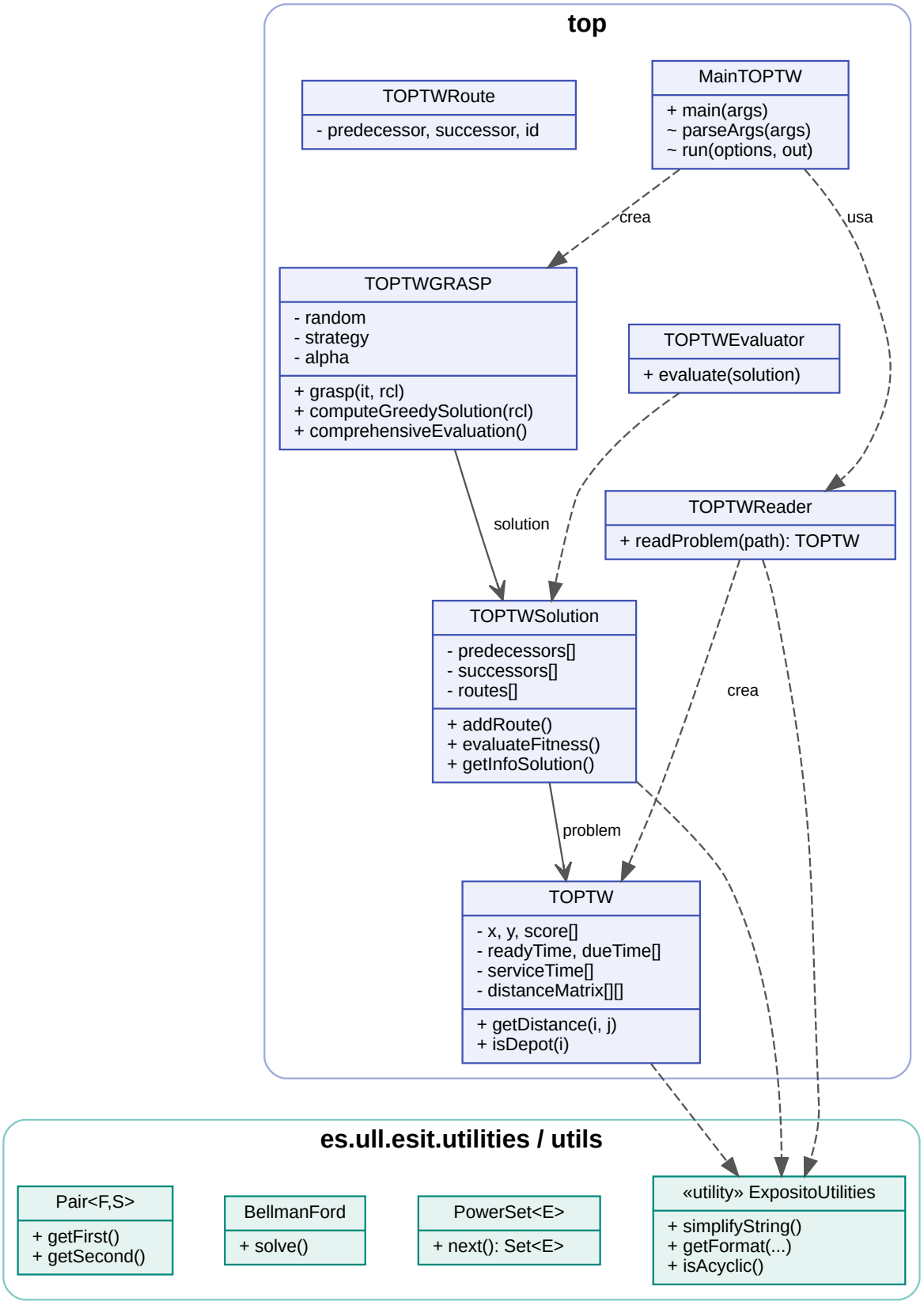
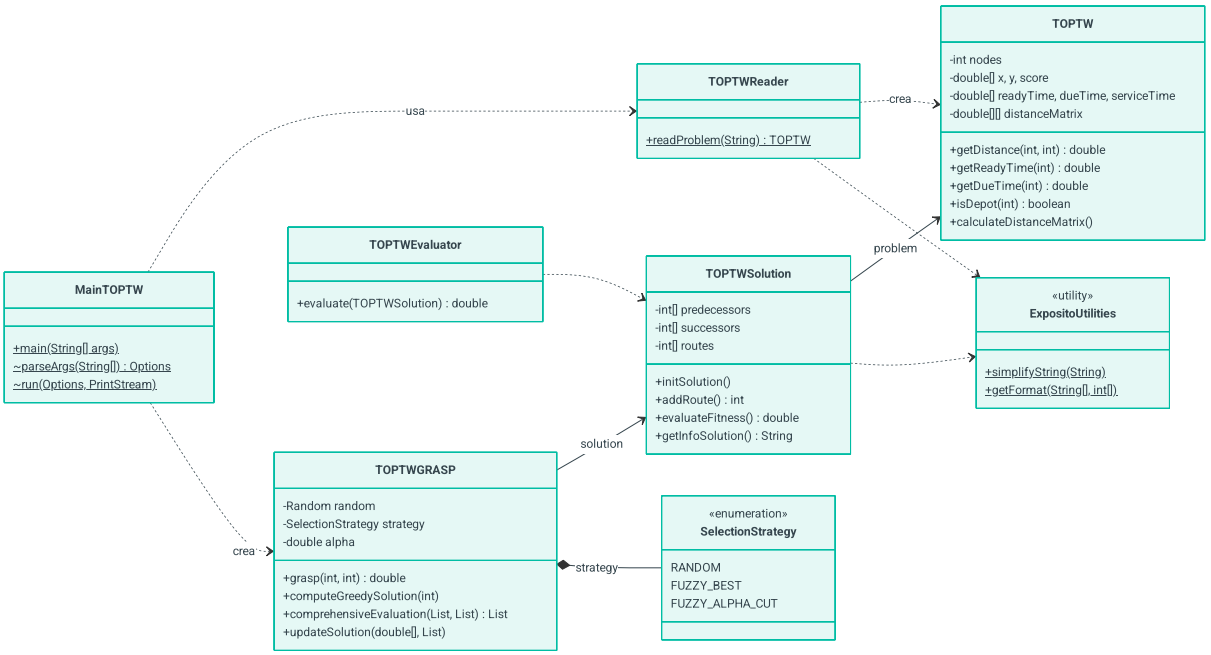


Diagrama de clases



Responsabilidades

`TOPTW` — Datos de la instancia. Guarda en vectores indexados por nodo las coordenadas, la puntuación, la ventana temporal y la duración de la visita, y precalcula la matriz de distancias euclídeas. Cualquier índice mayor que el número de POI se considera un depósito virtual y se traduce internamente al nodo 0.

`TOPTWReader` — Convierte un fichero de instancia en un objeto `TOPTW`. Lanza `IOException` ante ficheros inexistentes, truncados o mal formados (antes llamaba a `System.exit(0)`).

`TOPTWSolution` — Representa las rutas con dos vectores, `predecessors` y `successors`, que forman una lista doblemente enlazada circular por ruta. `routes[k]` guarda el depósito de la ruta *k*. Insertar un POI tras otro es O(1).

`TOPTWGRASP` — Fase constructiva del GRASP (ver Algoritmo). El generador aleatorio y el flujo de salida se inyectan en el constructor, lo que permite tests reproducibles y usar la clase sin consola.

`TOPTWEvaluator` — Calcula la función objetivo (suma de puntuaciones) y la guarda en la solución.

`MainTOPTW` — Interpreta los argumentos y ejecuta el experimento.

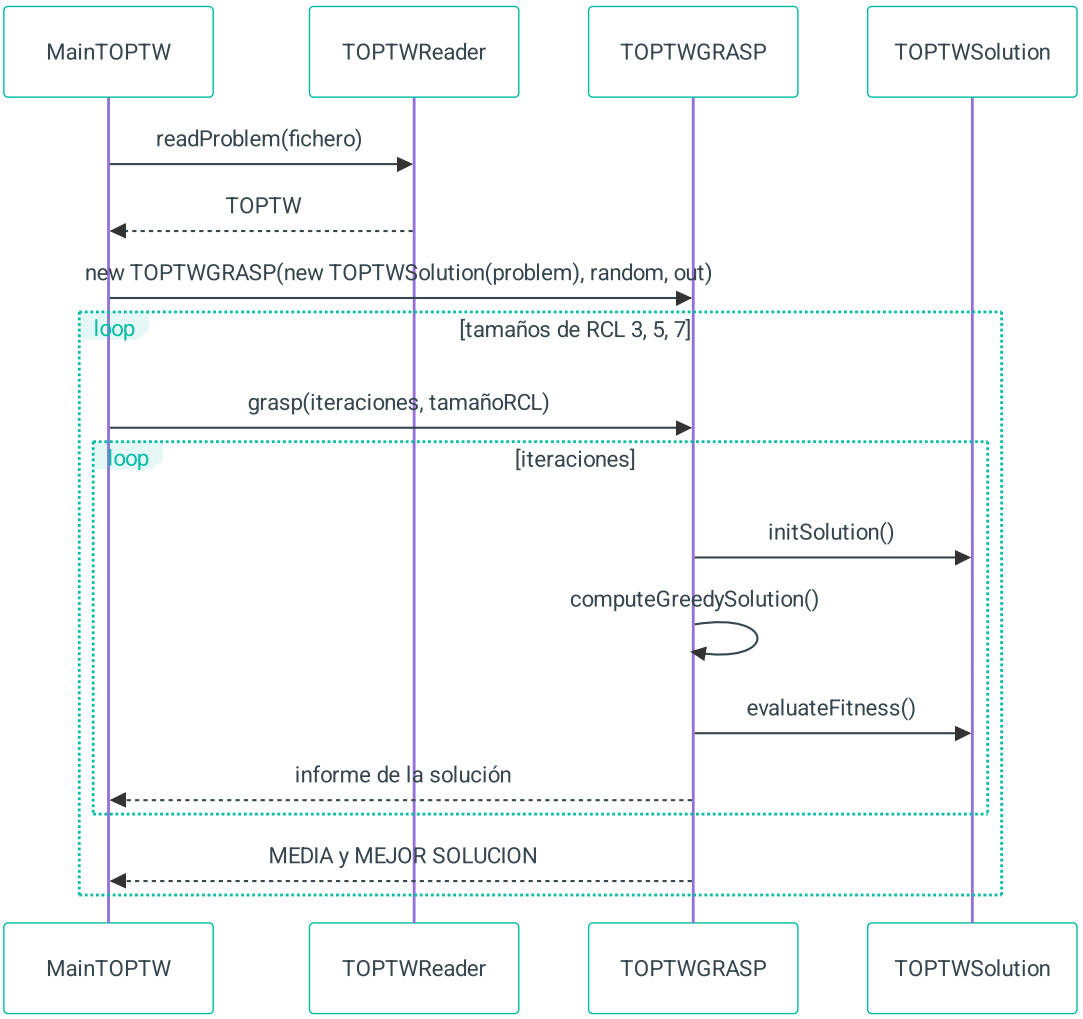
Representación de una solución

Con 100 POI y 2 rutas, `0 → 5 → 3 → 0` y `101 → 43 → 101` se almacenan así:

Nodo	0	3	5	43	101
`successors`	5	0	3	101	43
`predecessors`	3	5	0	101	43

`routes = [0, 101, -1, …]`. Recorrer la ruta *k* es seguir `successors` desde `routes[k]` hasta volver a él.

Flujo de una ejecución



Dependencias externas

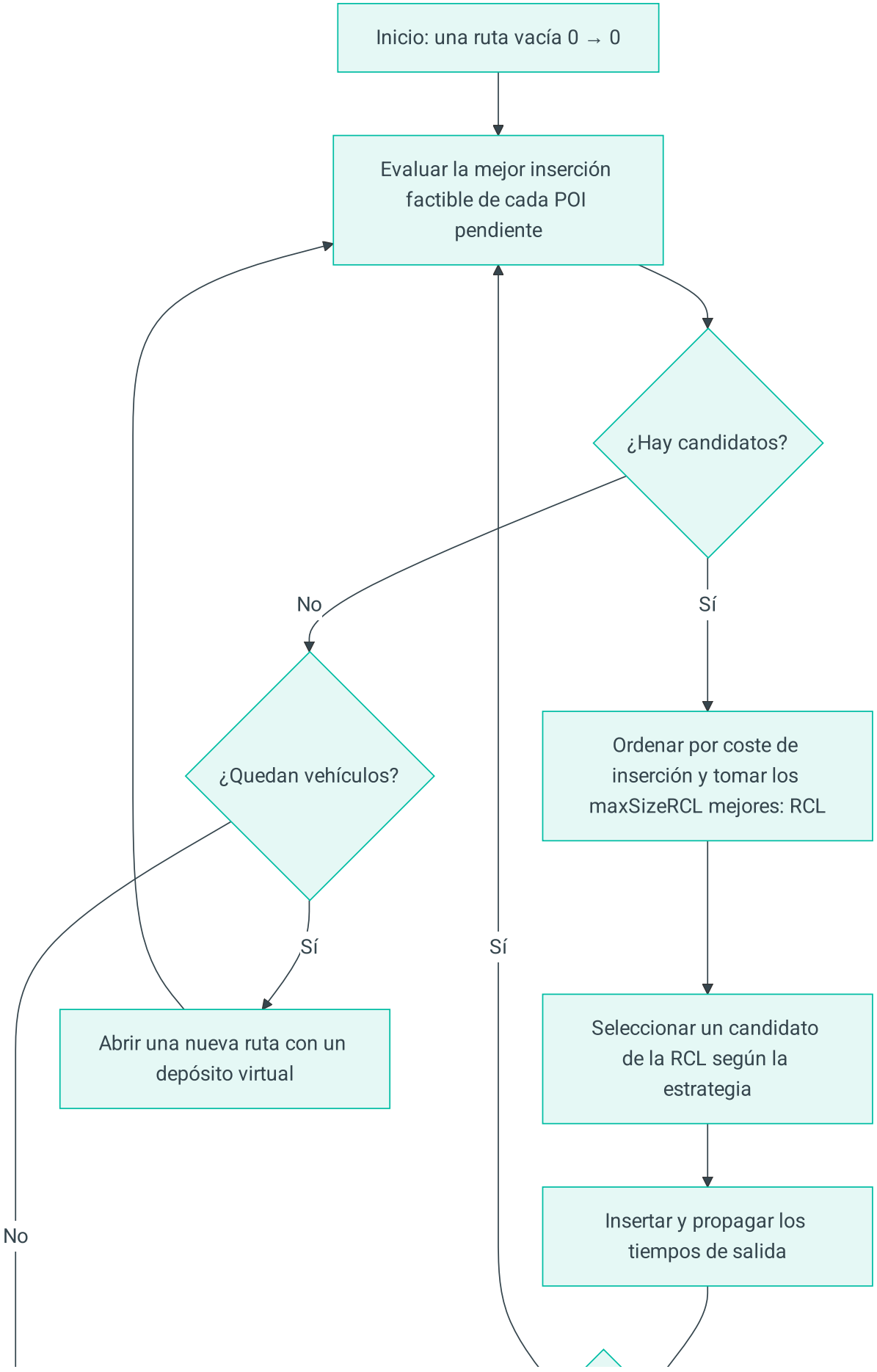
En ejecución, ninguna. En construcción, JUnit 4.13.2 (tests) y los plugins de Maven descritos en Plugins configurados.

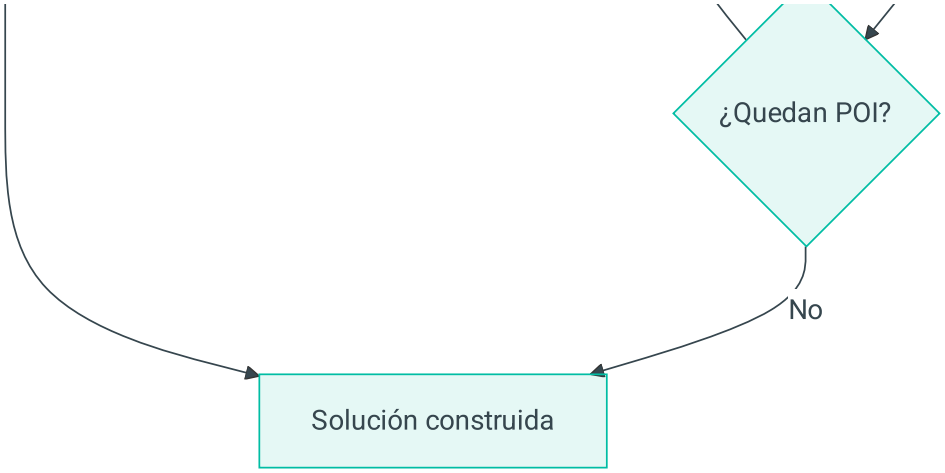
Algoritmo GRASP

El problema

Se dispone de *m* rutas (días). Cada ruta sale del depósito en el instante 0 y debe volver antes de `T_max` (cierre de la ventana del depósito). Cada POI *i* tiene una puntuación `S_i`, una duración de visita `d_i` y una ventana `[O_i, C_i]`: la visita puede empezar como pronto en `O_i` (si se llega antes, se espera) y debe empezar estrictamente antes de `C_i`. Se busca un conjunto de rutas que **maximice la suma de puntuaciones** visitando cada POI como mucho una vez.

Esquema general


Inicio: una ruta vacía 0 → 0
Evaluar la mejor inserción factible de cada POI pendiente
¿Hay candidatos?
No
Sí
¿Quedan vehículos?
Ordenar por coste de inserción y tomar los maxSizeRCL mejores: RCL
Sí
Sí
Abrir una nueva ruta con un depósito virtual
Seleccionar un candidato de la RCL según la estrategia
Insertar y propagar los tiempos de salida
No



`grasp(maxIterations, maxSizeRCL)` repite esta construcción `maxIterations` veces y registra la puntuación media y la mejor. El esqueleto deja preparado un punto de extensión para una fase de búsqueda local, que el proyecto original no implementa.

Evaluación de una inserción

`comprehensiveEvaluation` prueba cada POI pendiente en cada posición de cada ruta abierta. Insertar el candidato *c* entre `pre` y `suc` es factible si:

1. Se llega a *c* antes de su cierre: `salida(pre) + dist(pre, c) < C_c`.
2. La salida de *c*, `max(llegada, O_c) + d_c`, no supera `T_max`.
3. Tras el retraso que provoca la inserción, cada nodo posterior de la ruta, hasta volver al depósito, sigue cumpliendo su ventana y la duración máxima.

El **coste** de la inserción es la hora de regreso al depósito. Para cada POI se conserva su posición de menor coste; el resultado es la lista de candidatos `{cliente, ruta, predecesor, coste, puntuación}`.

En la versión refactorizada, la comprobación de un nodo está en `departureTime()` y la de una posición completa en `insertionCost()`, lo que reduce la complejidad cognitiva de `comprehensiveEvaluation` sin cambiar el resultado.

Lista restringida de candidatos (RCL)

Los candidatos se ordenan por coste creciente y la RCL la forman los `maxSizeRCL` primeros. El experimento usa tamaños 3, 5 y 7: cuanto mayor, más diversidad y menos voracidad.

Estrategias de selección

Estrategia	Regla
`RANDOM`	Uniforme sobre la RCL
`FUZZY_BEST`	El candidato con menor grado de pertenencia μ (mayor puntuación)
`FUZZY_ALPHA_CUT` *(por defecto)*	Uniforme entre los candidatos con μ ≤ α; si no hay ninguno, uniforme sobre la RCL

El grado de pertenencia borroso de un candidato es

$$\mu(c) = 1 - S_c / \max_i S_i$$

de modo que los POI más interesantes tienen μ cercano a 0. Con α = 0,8 solo se eligen POI cuya puntuación sea al menos el 20 % de la máxima, siempre que haya alguno en la RCL. Así la construcción combina el criterio temporal (la RCL se forma por coste) con el interés turístico (la selección favorece la puntuación).

Actualización tras insertar

`updateSolution` enlaza el POI en la lista doblemente enlazada y recorre la ruta desde el predecesor recalculando los tiempos de salida de los nodos siguientes:

```
t ← salida(pre)
repetir
    suc ← sucesor(pre)
    t ← max(t + dist(pre, suc), O_suc) + d_suc
    salida(suc) ← t
    pre ← suc
hasta volver al depósito
```

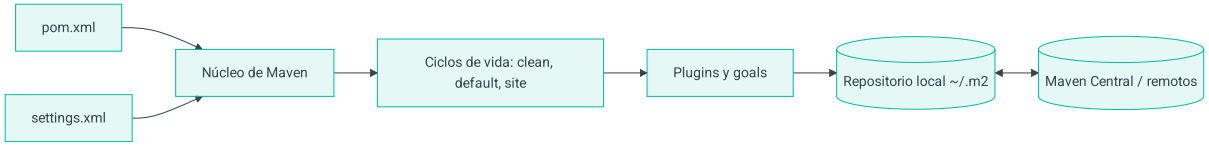
Complejidad

Cada evaluación completa cuesta O(P · L · L) en el peor caso (P POI pendientes, L longitud de las rutas), y se repite tras cada inserción, por lo que una construcción cuesta del orden de $O(P^2 \cdot L^2)$. Para las instancias de 100 POI una construcción tarda unos milisegundos.

Maven y su ciclo de vida

Apache Maven es una herramienta de **gestión y construcción de proyectos** basada en el concepto de *Project Object Model* (POM). En lugar de describir *cómo* construir el proyecto (como hace Ant con sus `build.xml`), el `pom.xml` describe *qué* es el proyecto: sus coordenadas, dependencias y plugins. Maven aplica convenciones para todo lo demás.

Arquitectura



Elemento	Descripción
POM	Fichero XML con el modelo del proyecto. Hereda del *Super POM*, que define las convenciones por defecto.
Coordenadas	`groupId:artifactId:version` identifican cada artefacto de forma única (`es.ull.esit:expositotop:1.0.0-SNAPSHOT`).
Repositorios	El *local* (`~/.m2/repository`) actúa de caché; los *remotos* (Maven Central) contienen dependencias y plugins.
Plugins y goals	Todo el trabajo lo hacen plugins. Cada plugin ofrece *goals* (`compiler:compile`, `surefire:test`…).
Ciclos de vida	Secuencias ordenadas de fases. Cada fase tiene goals asociados (*bindings*).
Dependencias	Se declaran con su *scope* (`compile`, `test`, `provided`, `runtime`…) y Maven resuelve también las transitivas.
Perfiles	Configuración alternativa que se activa con `-P` (aquí, `docs`).

Estructura estándar de directorios

```
ExpositoTOP/
├── pom.xml
├── src/
│   ├── main/java/          ← código de producción (antes: src/)
│   ├── main/resources/
│   ├── test/java/          ← tests JUnit (nuevos)
│   └── test/resources/     ← instancia mini.txt para tests
├── Instances/              ← datos de entrada del recomendador
└── target/                 ← salida de la construcción (no se versiona)
```

Los tres ciclos de vida

Ciclo	Fases principales	Uso
`clean`	`pre-clean`, `clean`, `post-clean`	Borrar `target/`
`default`	`validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`	Construir y distribuir
`site`	`pre-site`, `site`, `post-site`, `site-deploy`	Generar el sitio de informes

Ejecutar una fase ejecuta **todas las anteriores** de su ciclo: `mvn install` valida, compila, prueba, empaqueta y verifica antes de instalar.

Fases del ciclo `default` en este proyecto

Fase	Goals ejecutados (según el `pom.xml`)	Resultado
`validate`	`enforcer:enforce`	Comprueba Java ≥ 17, Maven ≥ 3.6.3 y la convergencia de dependencias
`compile`	`resources:resources`, `compiler:compile`	`target/classes` (con `-Xlint:all`)
`test`	`jacoco:prepare-agent`, `compiler:testCompile`, `surefire:test`	43 tests ejecutados; `target/surefire-reports`, `target/jacoco.exec`
`package`	`jar:jar`, `source:jar-no-fork`, `javadoc:jar`	`ExpositoTOP.jar` ejecutable, `-sources.jar`, `-javadoc.jar`
`verify`	`jacoco:report`, `jacoco:check`, `checkstyle:check`, `pmd:check`, `pmd:cpd-check`, `spotbugs:check`	Informe de cobertura (mínimo 70 % de líneas), informes de estilo y análisis estático
`install`	`install:install`	Copia el JAR y el POM a `~/.m2/repository/es/ull/esit/expositotop/`
`deploy`	`deploy:deploy`	Publica en el repositorio de `distributionManagement` (aquí, `target/deploy-repo`)

Sobre `deploy`

`deploy` necesita un repositorio remoto. Para que pueda probarse sin servidor, el `pom.xml` define un repositorio en disco (`file://…/target/deploy-repo`). Para publicar en GitHub Packages basta con `mvn deploy -DaltDeploymentRepository=github::https://maven.pkg.github.com/USUARIO/ExpositoTOP` y un *token* en `~/.m2/settings.xml`.

Migración del proyecto original

El proyecto se entregó como proyecto **NetBeans/Ant** (`build.xml`, `nbproject/`) con restos de una configuración de Eclipse que apuntaba a Maven (`.classpath` con `MAVEN2_CLASSPATH_CONTAINER`, `.settings/org.eclipse.m2e.core.prefs`) pero **sin** `pom.xml`. También incluía artefactos compilados

(`build/`, `dist/*.jar`, `dist/javadoc`) y un fichero de copia de seguridad (`ExpositoUtilities.java~`).

Pasos realizados (ver ADR 0001):

1. Mover `src/` a `src/main/java/` conservando los paquetes.
2. Crear `pom.xml` con coordenadas, Java 17, codificación UTF-8 y plugins.
3. Añadir `src/test/java` con tests JUnit.
4. Eliminar `build/`, `dist/`, `nbproject/`, ficheros de IDE y la copia `*~`; añadir `.gitignore`.
5. Renombrar `mainTOPTW` a `MainTOPTW` (convención de nombres de clase) y declararla como `Main-Class` del JAR.

Órdenes de Maven y uso desde el IDE

Instalación y verificación

Linux / macOS

```
# Opción 1: gestor de paquetes
sudo apt install maven          # Debian/Ubuntu
brew install maven              # macOS
# Opción 2: binario oficial
wget https://dlcdn.apache.org/maven/maven-3/3.9.11/binaries/apache-maven-3.9.11-
bin.tar.gz
tar xzf apache-maven-3.9.11-bin.tar.gz -C /opt
export PATH=/opt/apache-maven-3.9.11/bin:$PATH
```

Windows

1. Descargar el `.zip` binario de https://maven.apache.org/download.cgi.
2. Descomprimir en `C:\Program Files\Apache\maven`.
3. Definir `JAVA_HOME` y añadir `C:\Program Files\Apache\maven\bin` al `PATH`.

Verificación:

```
$ mvn -v
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)
Maven home: /opt/maven
Java version: 21.0.12.1, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.18.44", arch: "amd64", family: "unix"
```

Una prueba básica es generar un proyecto de ejemplo con un arquetipo y construirlo:

```
mvn archetype:generate -DgroupId=es.ull.prueba -DartifactId=hola \
    -DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.5 -
DinteractiveMode=false
cd hola && mvn package && java -cp target/hola-1.0-SNAPSHOT.jar es.ull.prueba.App
```

Órdenes principales (línea de órdenes)

Orden	Qué hace
`mvn validate`	Valida el POM y ejecuta las reglas del enforcer
`mvn compile`	Compila el código de producción
`mvn test`	Compila y ejecuta los tests
`mvn test -Dtest=TOPTWReaderTest`	Ejecuta una sola clase de test (`#método` para un método)
`mvn package`	Genera los JAR en `target/`
`mvn package -DskipTests`	Empaqueta sin ejecutar los tests
`mvn verify`	Además, cobertura y análisis estático
`mvn install`	Instala el artefacto en el repositorio local
`mvn deploy`	Publica el artefacto en el repositorio remoto configurado
`mvn clean`	Borra `target/` (se combina: `mvn clean install`)
`mvn site`	Genera el sitio de informes en `target/site/`
`mvn exec:java`	Ejecuta `MainTOPTW` con los argumentos de `exec.args`
`mvn verify sonar:sonar`	Análisis de calidad en SonarQube
`mvn -Pdocs package`	Construcción más documentación (Doxygen + MkDocs)
`mvn dependency:tree`	Árbol de dependencias
`mvn help:effective-pom`	POM efectivo (con lo heredado del Super POM)
`mvn versions:display-plugin-updates`	Plugins con versiones más recientes

Opciones globales útiles: `-q` (silencioso), `-X` (depuración), `-o` (sin conexión), `-B` (no interactivo, para CI), `-U` (forzar actualización de *snapshots*), `-T 1C` (un hilo por núcleo), `-D propiedad=valor` y `-P perfil`.

Uso desde el IDE

IntelliJ IDEA

1. **File** → **Open** y seleccionar el `pom.xml` → *Open as Project*. IntelliJ importa el proyecto, descarga dependencias y marca `src/main/java` y `src/test/java`.
2. Panel **Maven** (lateral derecho): *Lifecycle* contiene las fases (`validate` … `deploy`); doble clic para ejecutarlas. *Plugins* lista los goals (`exec:java`, `sonar:sonar`, `jacoco:report`…).
3. **Run** → **Edit Configurations** → **+** → **Maven** para guardar órdenes con argumentos (por ejemplo `verify sonar:sonar -Dsonar.token=...`).
4. Botón *Reload All Maven Projects* tras modificar el `pom.xml`.

Eclipse

1. **File** → **Import** → **Maven** → **Existing Maven Projects** y seleccionar la carpeta.
2. Clic derecho en el proyecto → **Run As**: `Maven build…` (permite escribir los goals, p. ej. `clean verify`), `Maven install`, `Maven test`, `Maven clean`.
3. Clic derecho → **Maven** → **Update Project** tras cambiar el `pom.xml`.

NetBeans

1. **File** → **Open Project** y elegir la carpeta (NetBeans reconoce el `pom.xml`).
2. Botones *Build* (`install`), *Clean and Build* (`clean install`), *Test* y *Run*.
3. Clic derecho → **Run Maven** → **Goals…** para cualquier fase o goal.

VS Code

Con el *Extension Pack for Java*: vista **Maven** en el explorador, clic derecho sobre el proyecto o sobre una fase del *Lifecycle* para ejecutarla.

Pruebas del ciclo de vida

La siguiente tabla resume lo que produce cada fase de este proyecto. Las capturas desde la consola y desde el IDE deben incluirse en el informe.

Fase	Comprobación
`mvn validate`	`BUILD SUCCESS` sin compilar nada; falla si se usa un JDK < 17
`mvn compile`	`target/classes/top/*.class`; sin avisos de `-Xlint`
`mvn test`	`Tests run: 43, Failures: 0, Errors: 0, Skipped: 0`
`mvn package`	`target/ExpositoTOP.jar`; `java -jar target/ExpositoTOP.jar --iterations 10 c101.txt` funciona
`mvn verify`	`target/site/jacoco/index.html`, `target/checkstyle-result.xml`, `target/pmd.xml`, `target/spotbugsXml.xml`
`mvn install`	`~/.m2/repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/`
`mvn deploy`	`target/deploy-repo/snapshots/es/ull/esit/expositotop/1.0.0-SNAPSHOT/` con `maven-metadata.xml`

Plugins configurados

Todos los plugins tienen la versión fijada en `<properties>` para que la construcción sea reproducible (sin versión, Maven usa la que define el Super POM de la versión de Maven instalada, que cambia entre versiones).

Construcción

Plugin	Fase	Propósito y configuración
maven-enforcer-plugin	`validate`	Impone **requisitos del entorno**: Java ≥ 17, Maven ≥ 3.6.3 y `dependencyConvergence` (todas las rutas de dependencias transitivas deben resolver a la misma versión). Evita errores difíciles de diagnosticar en máquinas mal configuradas.
maven-compiler-plugin	`compile`, `test-compile`	Compila con `<release>17</release>` (garantiza que solo se usa la API de Java 17 aunque se compile con un JDK posterior) y `-Xlint:all` con `showWarnings`, de modo que cualquier aviso del compilador aparece en la salida.
maven-surefire-plugin	`test`	Ejecuta los tests JUnit. `workingDirectory` = raíz del proyecto, porque los tests leen `Instances/TOPTW/*.txt` con rutas relativas; `redirectTestOutputToFile` evita volcar la salida del GRASP en la consola.
maven-jar-plugin	`package`	Genera `ExpositoTOP.jar` con `Main-Class: top.MainTOPTW` en el manifiesto (se ejecuta con `java -jar`) y entradas `Implementation-*`.
maven-source-plugin	`package`	Adjunta `ExpositoTOP-sources.jar` (necesario para publicar en repositorios y útil para los IDE).
maven-javadoc-plugin	`package`	Adjunta `ExpositoTOP-javadoc.jar`. `doclint` desactivado porque los comentarios usan comandos de Doxygen (`\brief`), que Javadoc trata como texto.
maven-install-plugin / maven-deploy-plugin	`install` / `deploy`	Versiones fijadas. El destino de `deploy` lo define `distributionManagement`.
exec-maven-plugin	bajo demanda	`mvn exec:java` ejecuta `MainTOPTW` en la JVM de Maven con `exec.args`. En el perfil `docs`, `exec:exec` lanza `doxygen` y `mkdocs build` en la fase `package`.

Calidad

Plugin	Fase	Propósito y configuración
jacoco-maven-plugin	`test`, `verify`	`prepare-agent` instrumenta la JVM de los tests; `report` genera HTML, CSV y **XML** (`target/site/jacoco/jacoco.xml`, el que lee SonarQube para la cobertura); `check` **rompe la construcción** si la cobertura de líneas baja del 70 %.
maven-checkstyle-plugin	`verify`	Comprueba la guía de estilo de Google (`google_checks.xml`). Configurado como **informativo** (`failOnViolation=false`): el código mantiene la indentación de 4 espacios del proyecto original y Google exige 2.
maven-pmd-plugin	`verify`	`pmd:check` (reglas de buenas prácticas, errores probables, código muerto) y `pmd:cpd-check` (código duplicado de 100 o más *tokens*). Informativo.
spotbugs-maven-plugin	`verify`	Busca patrones de error en el *bytecode* (nulos, recursos sin cerrar, exposición de representación interna…) con esfuerzo máximo. Informativo.
sonar-maven-plugin	bajo demanda	`mvn sonar:sonar` envía el análisis a SonarQube. Las propiedades `sonar.*` del POM fijan clave, URL, versión de Java, exclusiones y la ruta del informe de JaCoCo.

Informes (`mvn site`)

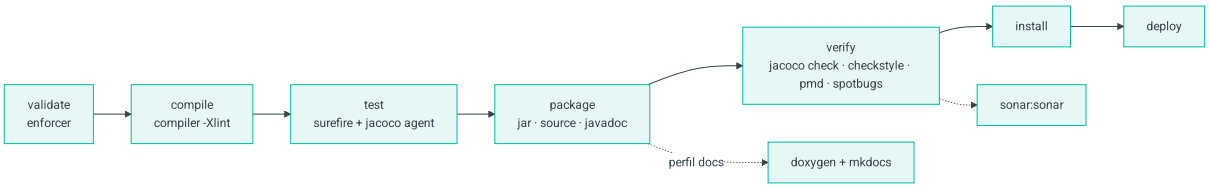
La sección `<reporting>` añade al sitio de Maven los informes de proyecto (`project-info-reports`), JaCoCo, Checkstyle, PMD/CPD, SpotBugs y Javadoc.

Perfil `docs`

```
mvn -Pdocs package
```

Ejecuta, después de empaquetar, `doxygen Doxyfile` y `mkdocs build`. Integra la documentación en la construcción, siguiendo el enfoque *Docs-as-Code*: la documentación se versiona y se construye junto al código.

Cómo contribuye cada plugin



Los plugins de calidad convierten la construcción en un **control de calidad automático**: cada `mvn verify` (en local o en GitHub Actions) mide la cobertura, el estilo y los defectos probables, y SonarQube agrega esas métricas en el tiempo.

Análisis de calidad con SonarQube

SonarQube realiza **análisis estático** del código: detecta *bugs*, vulnerabilidades, *security hotspots* y *code smells*, mide duplicaciones, complejidad y cobertura de tests, y decide si el proyecto supera una **Quality Gate** (umbral de calidad). Con Maven, el análisis lo ejecuta el `sonar-maven-plugin`, que aprovecha la información del POM (fuentes, binarios, dependencias y versión de Java).

1. Levantar SonarQube

Docker (recomendado)

```
docker compose -f sonarqube/docker-compose.yml up -d
# o bien, sin compose:
docker run -d --name sonarqube -p 9000:9000 sonarqube:community
```

Abrir http://localhost:9000, entrar con `admin` / `admin` y cambiar la contraseña.

Distribución ZIP

1. Descargar *SonarQube Community Build* de https://www.sonarsource.com/products/sonarqube/downloads/.
2. Descomprimir y ejecutar `bin/linux-x86-64/sonar.sh start` (o `bin\windows-x86-64\StartSonar.bat`).
3. Requiere Java 17 o 21 y, en Linux, `vm.max_map_count ≥ 524288`.

SonarQube Cloud

Para el repositorio de GitHub se puede usar SonarQube Cloud (gratuito para proyectos públicos): importar el repositorio en https://sonarcloud.io, crear el secreto `SONAR_TOKEN` en GitHub y fijar las variables `SONAR_ORGANIZATION` y `SONAR_PROJECT_KEY`. El *workflow* `.github/workflows/ci.yml` ya ejecuta el análisis cuando el secreto existe.

2. Crear el proyecto y el token

1. **Projects** → **Create project** → **Local project**. Clave: `ExpositoTOP`.
2. **Analysis method** → **Locally** → **Generate a token** (o *My Account* → *Security* → *Generate Tokens*, tipo *Project Analysis Token*). Guardar el token.

3. Configuración en el `pom.xml`

```
<sonar.projectKey>ExpositoTOP</sonar.projectKey>
<sonar.projectName>ExpositoTOP - RTVRPTW</sonar.projectName>
<sonar.host.url>http://localhost:9000</sonar.host.url>
<sonar.sourceEncoding>UTF-8</sonar.sourceEncoding>
<sonar.java.source>17</sonar.java.source>
<sonar.coverage.jacoco.xmlReportPaths>${project.build.directory}/site/jacoco/jacoco.
xml</sonar.coverage.jacoco.xmlReportPaths>
<sonar.exclusions>Instances/**,docs/**,site/**</sonar.exclusions>
```

Y el plugin, con versión fijada en `pluginManagement`:

```
<plugin>
    <groupId>org.sonarsource.scanner.maven</groupId>
    <artifactId>sonar-maven-plugin</artifactId>
    <version>${sonar-maven-plugin.version}</version>
</plugin>
```

> **No guardar el token en el POM**
>
> El token es una credencial: se pasa por la línea de órdenes o por la variable de entorno `SONAR_TOKEN`, nunca en el repositorio.

4. Ejecutar el análisis

```
mvn clean verify sonar:sonar -Dsonar.token=TOKEN
```

`verify` es necesario antes: genera los `.class` (SonarQube Java los necesita para el análisis semántico) y el informe XML de JaCoCo con la cobertura. Al terminar, el log muestra `ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=ExpositoTOP`.

Desde el IDE: crear una configuración de ejecución Maven con `clean verify sonar:sonar -Dsonar.token=TOKEN`. Con **SonarQube for IDE** (antes SonarLint) y el modo *Connected Mode* se ven las mismas incidencias mientras se edita.

5. Análisis del código original

Para comparar el antes y el después, se analiza también la versión original (etiqueta `v0-original` del repositorio). Como no tenía `pom.xml`, se compila con `javac` y se usa el escáner independiente (*SonarScanner CLI*) en Docker:

```
git worktree add ../ExpositoTOP-original v0-original
cd ../ExpositoTOP-original
mkdir -p out && javac -d out $(find src -name "*.java")
docker run --rm --network host -v "$PWD:/usr/src" sonarsource/sonar-scanner-cli \
  -Dsonar.host.url=http://localhost:9000 -Dsonar.token=TOKEN \
  -Dsonar.projectKey=ExpositoTOP-original -Dsonar.sources=src \
  -Dsonar.java.binaries=out -Dsonar.exclusions="**/*~"
```

Después se analiza la rama `main` con Maven (paso 4) y se comparan los dos proyectos.

6. Qué mirar en el panel

Sección	Qué indica
Quality Gate	*Passed/Failed* según las condiciones de *Sonar way* sobre el código nuevo (cobertura ≥ 80 %, duplicación ≤ 3 %, sin incidencias nuevas, *hotspots* revisados)
Reliability / Security / Maintainability	Incidencias por cualidad del software y su calificación A–E
Security Hotspots	Código sensible que debe revisar una persona (aquí, el uso de `Random`)
Coverage	Cobertura de líneas y condiciones a partir del XML de JaCoCo
Duplications	Bloques y líneas duplicadas
Measures → Complexity	Complejidad ciclomática y cognitiva por fichero y método
Technical Debt	Tiempo estimado para corregir los *code smells*

Las incidencias encontradas y cómo se han resuelto se detallan en Incidencias y soluciones.

Incidencias detectadas y soluciones

Inventario de incidencias del **código original** según las reglas del perfil *Sonar way* para Java, agrupadas por tipo, y la solución aplicada en cada caso. Cada regla se identifica con su clave (`java:Sxxxx`) para localizarla en SonarQube (*Rules*). Las líneas se refieren a los ficheros originales.

Cómo se ha elaborado

Análisis con SonarQube Community Build (servidor 13.7) mediante `mvn verify sonar:sonar`, completado con los informes de SpotBugs, PMD y Checkstyle que genera `mvn verify` y con la revisión del código. Las líneas se refieren a los ficheros originales.

Fiabilidad (*bugs*)

#	Regla	Dónde	Problema	Solución
B1	`java:S2259` Null pointers should not be dereferenced	`ExpositoUtilities.printFile` (l. 159-160)	Si `new FileReader` falla, `reader` es `null` y el `finally` lanza `NullPointerException`, ocultando la excepción original	`try-with-resources`; el método propaga `IOException`
B2	`java:S2095` Resources should be closed	`ExpositoUtilities.writeTextToFile` (l. 198-201)	Si `write` lanza una excepción, el `BufferedWriter` no se cierra	`try-with-resources`
B3	`java:S2119` Random objects should be reused	`TOPTWGRASP.aleatorySelectionRCL` (l. 71)	Se crea un `Random` en cada llamada (miles por iteración): ineficiente y de peor calidad estadística	Un único `Random` como campo, inyectable por constructor (ADR 0003)
B4	`java:S2272` Iterator.next() should throw NoSuchElementException	`PowerSet.next`	Al agotarse sigue devolviendo conjuntos, incumpliendo el contrato de `Iterator`	Lanza `NoSuchElementException` si `!hasNext()`
B4b	`java:S4348` iterator() should not return this	`PowerSet.iterator`	La clase era su propio iterador: solo podía recorrerse una vez	`PowerSet` es solo `Iterable` y cada `iterator()` crea un iterador interno nuevo

#	Regla	Dónde	Problema	Solución
B5	`java:S1201` / `java:S1206` equals/hashCode	`TOPTWSolution.equals(TOPTWSolution)` (l. 56)	Sobrecarga en lugar de sobrescribir `equals(Object)`: las colecciones la ignoran; además no hay `hashCode`	`equals(Object)` y `hashCode()` coherentes basados en `predecessors`
B6	`java:S2589` Gratuitous boolean expressions	`ExpositoUtilities.isAcyclic` (l. 350-351)	`acyclic` nunca cambia: condición siempre cierta	Bucle `for` simplificado

Defectos funcionales que SonarQube no detecta

El análisis estático no entiende el formato de los ficheros ni la semántica de la salida, pero la revisión del código y los tests encontraron estos errores, más graves que los anteriores:

#	Dónde	Problema	Solución
F1	`TOPTWReader.readProblem` (l. 37-38)	La ventana temporal se leía de las **columnas fijas 8 y 9**, pero en el formato el campo previo es una lista de longitud variable. En las 10 instancias `pr*` (con listas de longitud 2 y 4) se tomaban valores de la lista como horario: por ejemplo, el POI 25 de `pr01` quedaba con ventana `[2, 4]` en lugar de `[360, 505]`	Se usan los **dos últimos** campos de la línea (ADR 0002). Con semilla 42, la mejor puntuación en `pr01` pasa de **302** a **560**
F2	`TOPTWReader` (l. 43-45, 50-52)	Ante un error de lectura escribía el error y llamaba a `System.exit(0)`, es decir, terminaba con **código de éxito**	Lanza `IOException`; `MainTOPTW` registra el error y termina con código 1
F3	`TOPTWSolution.getInfoSolution` (l. 185)	La columna `SERVICE TIME` mostraba la duración del nodo **anterior**	Muestra la del nodo de la fila
F4	`TOPTWSolution.getInfoSolution` (l. 178-188)	En nodos que incumplen su ventana, las columnas de llegada y salida conservaban los valores de la fila anterior	Se muestran como `-`

Seguridad

#	Regla	Dónde	Valoración
H1	`java:S2245` (*Security Hotspot*) Pseudorandom number generators	`TOPTWGRASP` (l. 71)	`java.util.Random` no es criptográficamente seguro, pero aquí solo guía una heurística de optimización: se revisa y se marca como **Safe** en SonarQube. No hay vulnerabilidades.

Mantenibilidad (*code smells*)

Diseño y complejidad

#	Regla	Dónde	Solución
M1	`java:S3776` Cognitive Complexity > 15	`TOPTWGRASP.comprehensiveEvaluation` (l. 228-311, ~6 niveles de anidamiento), `computeGreedySolution`, `TOPTWSolution.getInfoSolution`	Extracción de `departureTime()`, `insertionCost()`, `sortedCandidates()`, `newDepartureTimes()`, `select()`; uso de `Math.max`. Comportamiento verificado con test de regresión
M1b	`java:S6206` Records should be used	`Pair`	`public record Pair<F, S>(F first, S second)` (Java 16)
M2	`java:S1871` Duplicated branches	`switch` de `computeGreedySolution` (l. 155 y 161)	`case 1` y `default` eran iguales; la estrategia pasa a ser un `enum` (ADR 0005)
M3	Selección fijada en el código (`int selection = 3`)	`computeGreedySolution` (l. 152-153)	`SelectionStrategy` y `alpha` configurables con *setters*
M4	`java:S4144` Methods should not have identical implementations	`TOPTW.getDistance(int,int)` y `getTime` (l. 112-122)	`getTime` delega en `getDistance`
M5	Duplicación de bloques	Comparador de candidatos (l. 133 y 188), inicialización de tiempos (l. 123 y 178)	Métodos `sortedCandidates()` y `newDepartureTimes()`
M6	`java:S1186` / `java:S125` Empty method / commented-out code	`TOPTWEvaluator.evaluate` (cuerpo entero comentado, l. 7-32)	Implementado: calcula y guarda la función objetivo

#	Regla	Dónde	Solución
M7	`java:S1118` Utility classes should not have public constructors	`ExpositoUtilities`, `TOPTWReader`	Clases `final` con constructor privado

Robustez y buenas prácticas

#	Regla	Dónde	Solución
M8	`java:S1147` Exit methods should not be called	`TOPTWReader` (l. 45, 52)	Excepciones (ver F2)
M9	`java:S106` Standard outputs should not be used directly	`TOPTWGRASP` (l. 50, 66, 67), `TOPTWReader` (l. 44, 51), `TOPTWSolution.printSolution`, `ExpositoUtilities.printFile`	Se escribe en un `PrintStream` inyectado; solo `main` usa `System.out` (ADR 0004)
M10	`java:S108` / `java:S2221` Empty catch / catching `Exception`	`isInteger`, `isDouble` (l. 333, 342)	Se captura `NumberFormatException` y se devuelve `false` explícitamente; comprobación de `null`
M11	`java:S1444` `public static` fields should be `final`	`TOPTWEvaluator.NO_EVALUATED`, `TOPTWGRASP.NO_EVALUATED`	`public static final` en el evaluador; el de GRASP no se usaba y se elimina
M12	`java:S1168` Return empty array instead of null	`multiplyMatrices` (l. 181)	`IllegalArgumentException` con mensaje si las dimensiones no son compatibles
M13	`java:S2384` Mutable members returned directly	`TOPTW.getScore()`, `TOPTWSolution.getPredecessors()/getSuccessors()`, `BellmanFord.getDistances()`	Se devuelven copias
M14	`java:S1144` Unused private method	`ExpositoUtilities.getFirstAppearance`	Eliminado

#	Regla	Dónde	Solución
M15	`java:S1481` / `java:S1854` Unused variables / dead stores	`newDepot` (l. 176), `symbols` (l. 245), `line = null; parts = null` (l. 23, 40), inicializaciones `= -1` y `= 0` sobrescritas	Eliminados
M16	`java:S1226` Parameters should not be reassigned	`TOPTW.getDistance(i, j)`, `getTime`, `getX` … (`i = 0`)	Método privado `dataIndex()`
M17	`java:S1643` String concatenation in loops	`TOPTW.toString`, `getInfoSolution`, `ExpositoUtilities.getFormat`	`StringBuilder`

Convenciones y estilo

#	Regla	Dónde	Solución
C1	`java:S101` Class names	`mainTOPTW`	`MainTOPTW`
C2	`java:S100` Method names	`TOPTWGRASP.GRASP`	`grasp`
C3	`java:S117` / `java:S3008` Local / static field names	`COLUMN_WIDTH`, `INSTANCE` (locales), `NO_EVALUATED` (estático no final)	`columnWidth`, eliminado, `final`
C4	`java:S1126` Return of boolean literal	`TOPTW.isDepot` (l. 42-45)	`return a > this.nodes;`
C5	`java:S1125` Boolean literals	`validFinalInsertion==true` (l. 286)	Eliminado al reestructurar
C6	`java:S1066` Mergeable if	`ExpositoUtilities.getFormat(String)` (l. 205-206)	Condición unida con `&&`
C7	`java:S1155` Use isEmpty()	`rclAlphaCut.size() > 0` (l. 108)	`!rclPos.isEmpty()`
C8	`java:S1488` Local variable returned immediately	`aleatorySelectionRCL` (l. 74-75)	`return random.nextInt(maxTRCL);`
C9	`java:S1604` Anonymous class → lambda	Comparadores (l. 133, 188)	`Comparator.comparingDouble(a -> a[COST])`
C10	`java:S2293` Diamond operator	`new ArrayList<double[]>()`, `new Pair<A, B>(…)`	`new ArrayList<>()`
C11	`java:S1319` Use interfaces in declarations	Parámetros y variables `ArrayList<…>`	`List<…>`
C12	`java:S1197` Array designators on the	`double a[][]` (l. 176, 186)	`double[][] a`

#	Regla type	Dónde	Solución
C13	`java:S1659` One declaration per line	`int pre=-1, suc=-1;` y similares	Una declaración por línea
C14	`java:S125` Commented-out code	`TOPTWGRASP` (l. 51-52, 57), `TOPTW` (l. 205)	Eliminado
C15	Javadoc vacío	`BellmanFord` (comentarios `/** */` sin texto)	Documentación Doxygen completa

Higiene del repositorio

Problema	Solución
Binarios y salidas de compilación versionados (`build/` , `dist/*.jar` , `dist/javadoc/`)	Eliminados; `.gitignore`
Copia de seguridad del editor `ExpositoUtilities.java~` con un paquete antiguo (`exposito.utilities`)	Eliminada
Configuración privada de IDE (`nbproject/private` , `.settings`)	Eliminada
Sin tests: cobertura 0 % (la *Quality Gate* exige ≥ 80 % en código nuevo)	43 tests JUnit (Pruebas)

Resumen antes / después (resultados reales)

Métrica	Original	Final
Quality Gate	Passed *(no evaluaba nada: primer análisis, sin código nuevo)*	**Passed** *(evaluando el código nuevo)*
Security (SonarQube)	0 · A	0 · A
Reliability (SonarQube)	**4 · D**	**0 · A**
Maintainability (SonarQube)	**73** · A	**0** · A
Coverage	**0,0 %** (793 líneas)	**97,6 %** (658 líneas)
Duplications	0,0 %	0,0 %
Tests JUnit	0	43
SpotBugs	22	0 *(10 `EI_EXPOSE_REP` excluidos y justificados en `spotbugs-exclude.xml`)*
PMD	44	0
Checkstyle (Google, avisos)	1625	informativo: estilo de 4 espacios frente a 2
Complejidad cognitiva máxima	95 (`comprehensiveEvaluation`)	< 15

Hallazgos durante el proceso

- **La Quality Gate aprueba sola el primer análisis**: solo evalúa el *código nuevo*, que no existe en la primera ejecución. Hay que mirar *Overall Code*.
- **Con menos de 20 líneas nuevas no se aplican las condiciones de cobertura ni duplicación**: tras corregir solo la fiabilidad (16 líneas) seguía en *Passed* con un 0 % de cobertura; al refactorizar el resto pasó a *Failed* hasta añadir los tests.
- **Falso positivo por idioma**: la regla de comentarios `TODO` detectó la palabra española «sobre *todo*».

- **SonarQube también analiza los tests**: marcó `assertFalse(a.equals(b))` (mejor `assertNotEquals`) y un `throws IOException` innecesario.
- **Perfil de calidad**: el servidor tenía por defecto el perfil *Sonar agentic AI*.
- **Las herramientas se solapan**: el `Random` creado en cada llamada lo detectaron SonarQube (`java:S2119`), SpotBugs (`DMI_RANDOM_USED_ONLY_ONCE`) y las inspecciones de IntelliJ.

Comportamiento conservado

El test `refactoringPreservesOriginalResults` comprueba que, con la misma semilla, el GRASP refactorizado obtiene las mismas puntuaciones que el original. Además se compararon 450 soluciones completas (rutas, puntuación y coste) de `c101`, `r101` y `rc101` con 50 iteraciones por tamaño de RCL: todas son idénticas.

Pruebas

El proyecto original no tenía tests. Se han añadido **43 tests JUnit 4** en `src/test/java`, que Maven ejecuta en la fase `test` y JaCoCo mide para la cobertura.

Clase de test	Qué verifica
`TOPTWReaderTest`	Lectura de `c101` y de una instancia mínima; **regresión de la ventana temporal en `pr01`**; errores con fichero inexistente, truncado o mal formado
`TOPTWTest`	Matriz de distancias euclídea y simétrica; traducción de depósitos virtuales; longitudes de rutas; copia defensiva; `toString`
`TOPTWSolutionTest`	Solución inicial; `addRoute`; fitness; informe (factible e infactible); `equals` / `hashCode`; copias
`TOPTWGRASPTest`	Factibilidad de las soluciones con las tres estrategias; reproducibilidad con semilla; **resultados idénticos al código original**; mejora en `pr01`; evaluación de inserciones; estrategias de selección
`MainTOPTWTest`	Interpretación de argumentos y ejecución de extremo a extremo
`TOPTWRouteTest`, `PairTest`	Accesores, igualdad, `hashCode`
`ExpositoUtilitiesTest`	Normalización de cadenas, formato, detección de números, producto de matrices, ciclos, ficheros
`PowerSetTest`, `BellmanFordTest`	2^n subconjuntos y contrato de `Iterator`; caminos mínimos con pesos negativos y camino máximo

Oráculo de factibilidad independiente

`TestSupport.assertFeasible` recorre cada ruta y comprueba, **sin usar el código del GRASP**, que:

- los enlaces `predecessors` / `successors` son coherentes,
- cada POI se visita como mucho una vez,
- se llega a cada nodo antes del cierre de su ventana,
- ninguna ruta supera la duración máxima.

Así los tests no se limitan a repetir la lógica que verifican.

Instancia mínima

`src/test/resources/mini.txt` tiene 4 POI y 2 rutas, con valores elegidos para poder calcular a mano los resultados (por ejemplo, insertar el POI 1 cuesta 10 + 5 + 10 = 25) e incluye un POI inalcanzable dentro de su ventana y una línea con lista de longitud 2.

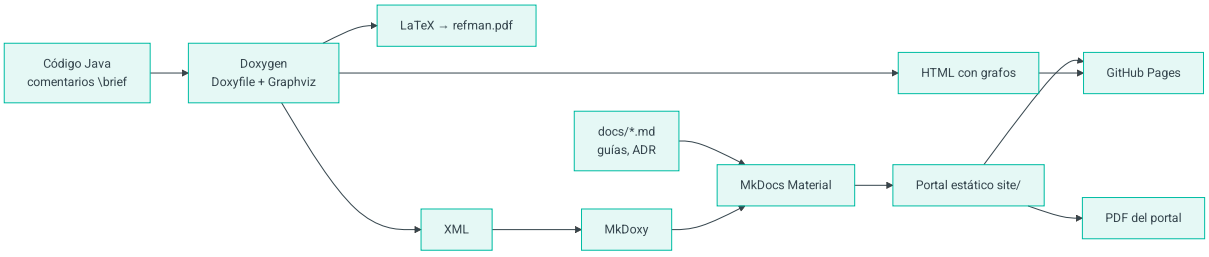
Ejecución

```
mvn test                                    # todos los tests
mvn test -Dtest=TOPTWGRASPTest              # una clase
mvn verify                                  # + informe de cobertura en
target/site/jacoco/index.html
```

Resultado esperado: `Tests run: 43, Failures: 0, Errors: 0, Skipped: 0`.

Documentación: Doxygen + MkDocs (Docs-as-Code)

La documentación sigue el paradigma **Docs-as-Code**: se escribe en texto plano (comentarios en el código y Markdown), se versiona en Git junto al código, se revisa en los *pull requests* y se construye y publica automáticamente en cada cambio.



1. Comentarios en el código

Todas las clases, métodos y atributos tienen un comentario con una **descripción breve** (`\brief`) y, cuando aporta, una **descripción detallada** (`\details`), además de `@param`, `@return` y `@throws`:

```
/**
 * \brief Coste de insertar un candidato justo después de un nodo de una ruta.
 *
 * \details Comprueba la ventana temporal del candidato, la del sucesor y la de
todos
 * los nodos siguientes hasta volver al depósito, así como la duración máxima de
ruta.
 *
 * @param routeIndex     ruta donde se inserta.
 * @param pre            nodo tras el que se inserta.
 * @param candidate      POI candidato.
 * @param departureTimes tiempos de salida por ruta y nodo.
 * @return tiempo de llegada al depósito tras la inserción, o NaN si no es factible.
 */
```

Cada paquete se documenta en su `package-info.java` y la página principal de Doxygen es `doxygen/mainpage.md`.

2. Doxyfile

Opciones relevantes (el fichero completo está en la raíz del repositorio):

Opción	Valor	Motivo
`OPTIMIZE_OUTPUT_JAVA`	`YES`	Presentación adaptada a Java (paquetes en lugar de *namespaces*)
`JAVADOC_AUTOBRIEF`	`YES`	Si falta `\brief`, la primera frase hace de resumen
`EXTRACT_ALL` / `EXTRACT_PRIVATE` / `EXTRACT_PACKAGE`	`YES`	Documenta también miembros privados y de paquete (documentación para mantenimiento)
`OUTPUT_LANGUAGE`	`Spanish`	Textos generados en castellano
`SOURCE_BROWSER`, `REFERENCED_BY_RELATION`, `REFERENCES_RELATION`	`YES`	Código navegable y referencias cruzadas
`HAVE_DOT`	**`YES`**	**Usa Graphviz para los grafos**
`CLASS_GRAPH`, `COLLABORATION_GRAPH`, `UML_LOOK`	`YES`	Diagramas de herencia y colaboración en estilo UML
`CALL_GRAPH`, `CALLER_GRAPH`	`YES`	Grafos de llamadas de cada método
`GRAPHICAL_HIERARCHY`, `DIRECTORY_GRAPH`	`YES`	Jerarquía de clases y dependencias entre directorios
`DOT_IMAGE_FORMAT` / `INTERACTIVE_SVG`	`svg` / `YES`	Grafos vectoriales con zoom
`GENERATE_LATEX`, `USE_PDFLATEX`, `PDF_HYPERLINKS`	`YES`	`refman.pdf`
`GENERATE_XML`	`YES`	Entrada para MkDoxy
`WARN_IF_UNDOCUMENTED`, `WARN_NO_PARAMDOC`, `WARN_LOGFILE`	`YES`, log	Detecta elementos sin documentar

```
doxygen Doxyfile                  # target/doxygen/{html,latex,xml}
make -C target/doxygen/latex      # target/doxygen/latex/refman.pdf
```

3. Portal MkDocs

`mkdocs.yml` configura el tema **Material** (navegación por pestañas, búsqueda en español, modo claro/oscuro, botón de copiar código, edición en GitHub), extensiones de Markdown (avisos, pestañas, diagramas Mermaid, fórmulas) y el plugin **MkDoxy**, que ejecuta Doxygen sobre `src/main/java` y genera páginas Markdown con la referencia de cada clase, integradas en la sección *Referencia API* de la navegación.

```
pip install -r requirements-docs.txt
mkdocs serve        # vista previa con recarga automática
mkdocs build        # site/
```

El HTML completo de Doxygen, con todos los grafos interactivos, se publica además en `/doxygen/` dentro del mismo portal.

4. Enriquecimiento con LLM

Las páginas de `docs/` (manual de uso, formato de instancias, preguntas frecuentes, arquitectura, algoritmo, guías de Maven y SonarQube, registros de decisiones) se generaron con un asistente basado en un modelo de lenguaje a partir del código y de los resultados de las pruebas. El flujo seguido fue:

1. Proporcionar al LLM el código fuente, el formato de las instancias y los resultados de la ejecución.
2. Pedir cada documento con un propósito y un público concretos (usuario final, mantenedor, evaluador).
3. **Revisar cada afirmación contra el código y los tests**: las salidas de ejemplo son ejecuciones reales y los valores numéricos (costes de inserción, puntuaciones) se comprueban en `TOPTWGRASPTest` y `TOPTWReaderTest`.
4. Versionar los `.md` y añadirlos a la navegación de `mkdocs.yml`.

El LLM acelera mucho la redacción, pero sin revisión puede inventar detalles plausibles (por ejemplo, salidas de ejemplo o versiones). Por eso la revisión humana, apoyada en tests, es parte del proceso.

5. Entregables generados

Entregable	Cómo se genera	Dónde
Portal web	`mkdocs build`	`site/`, publicado en GitHub Pages
HTML de Doxygen con grafos	`doxygen Doxyfile`	`target/doxygen/html`, publicado en `/doxygen/`
Manual de referencia PDF (Doxygen)	`make -C target/doxygen/latex`	`refman.pdf`, artefacto de GitHub Actions y en `/pdf/` del portal
PDF del portal	`python scripts/build_pdf.py`	`docs/pdf/ExpositoTOP-documentacion.pdf`

6. Publicación automática

El *workflow* `.github/workflows/docs.yml` instala Doxygen, Graphviz y LaTeX, genera todo lo anterior en cada *push* a `main` y lo despliega en **GitHub Pages** (*Settings → Pages → Source: GitHub Actions*).

Registro de decisiones de arquitectura (ADR)

Un ADR (*Architecture Decision Record*) documenta una decisión relevante: su contexto, la decisión tomada y sus consecuencias. Se escriben una vez y no se modifican; si una decisión cambia, se crea un ADR nuevo que sustituye al anterior.

Formato utilizado (Michael Nygard): **Estado · Contexto · Decisión · Consecuencias**.

ADR	Título	Estado
0001	Migración de NetBeans/Ant a Maven	Aceptada
0002	Leer la ventana temporal de los dos últimos campos	Aceptada
0003	Generador aleatorio único e inyectable	Aceptada
0004	Salida desacoplada de la consola	Aceptada
0005	Estrategia de selección de la RCL como `enum`	Aceptada
0006	Documentación Docs-as-Code con Doxygen y MkDocs	Aceptada

ADR 0001 — Migración de NetBeans/Ant a Maven

Estado: aceptada

Contexto

El proyecto se construía con el `build.xml` generado por NetBeans, dependiente de `nbproject/`. No declaraba dependencias ni versión de Java de forma portable, incluía binarios en el repositorio y no podía construirse fuera del IDE ni integrarse con SonarQube o con CI sin configuración manual.

Decisión

Adoptar Maven con la estructura estándar (`src/main/java`, `src/test/java`), Java 17 (LTS) y todos los plugins con la versión fijada. Eliminar los ficheros de Ant/NetBeans y los binarios. Renombrar `mainTOPTW` a `MainTOPTW` y usarla como `Main-Class` del JAR.

Consecuencias

- Construcción reproducible con una orden (`mvn verify`) en cualquier IDE y en CI.
- Integración directa con JaCoCo, SonarQube, Checkstyle, PMD y SpotBugs.
- Se pierde la configuración de ejecución de NetBeans, sustituida por `exec-maven-plugin`.
- Quien ejecute el proyecto necesita JDK 17 o superior (el enforcer lo comprueba).

ADR 0002 — Leer la ventana temporal de los dos últimos campos

Estado: aceptada

Contexto

Cada línea de nodo tiene el formato `i x y d S f a lista O C`, donde `lista` contiene `a` valores. `TOPTWReader` leía `O` y `C` de las posiciones fijas 8 y 9 (7 y 8 para el depósito), lo que solo es correcto si `a = 1`. En las 29 instancias de Solomon `a = 1`, pero en las 10 instancias `pr*` `a` toma los valores 1, 2 y 4. Por ejemplo, en `pr01`:

```
25 -54.755 14.368 4 14 1 4 1 2 4 8 360 505
```

se leía la ventana `[2, 4]` en lugar de `[360, 505]`, lo que hacía inalcanzables muchos POI.

Decisión

Tomar `O` y `C` de los **dos últimos** campos de la línea, válido para cualquier `a`, y convertir los errores de formato en `IOException` con un mensaje claro.

Consecuencias

- Los resultados de las instancias `pr*` cambian (y mejoran: en `pr01`, con semilla 42, la mejor puntuación pasa de 302 a 560). Los de Solomon no cambian.
- Un test de regresión (`readsTimeWindowFromLastFieldsWithVariableLengthList`) protege el comportamiento.

ADR 0003 — Generador aleatorio único e inyectable

Estado: aceptada

Contexto

`aleatorySelectionRCL` creaba un `new Random()` en cada llamada (regla `java:S2119`). Además de ser ineficiente, hacía imposible reproducir una ejecución y, por tanto, escribir tests de regresión que comprobaran que una refactorización no cambia el algoritmo.

Decisión

`TOPTWGRASP` recibe un `Random` en el constructor y lo reutiliza. El constructor de un argumento mantiene la compatibilidad usando `new Random()`. `MainTOPTW` acepta `--seed`.

Consecuencias

- Ejecuciones reproducibles, útiles para depurar, para comparar parámetros y para los tests.
- Con la misma semilla, el código refactorizado produce exactamente las mismas soluciones que el original, lo que se ha usado como prueba de equivalencia.
- El uso de `Random` sigue siendo un *Security Hotspot* (`java:S2245`) que se marca como seguro: no se usa con fines criptográficos.

ADR 0004 — Salida desacoplada de la consola

Estado: aceptada

Contexto

El algoritmo y el lector escribían directamente en `System.out` / `System.err` y el lector terminaba el proceso con `System.exit(0)` ante cualquier error (`java:S106` , `java:S1147`). Esto impedía usar las clases como biblioteca, probarlas sin capturar la consola y distinguir una ejecución correcta de una fallida (código de salida 0 en ambos casos).

Decisión

- `TOPTWGRASP` escribe en un `PrintStream` inyectado (o en ninguno, si es `null`) y expone `getLastAverageFitness()` y `getLastBestFitness()` .
- `TOPTWSolution.printSolution` y `ExpositoUtilities.printFile` reciben el `PrintStream` .
- `TOPTWReader` lanza `IOException` .
- Solo `MainTOPTW.main` usa `System.out` y `System.exit(1)` , marcados como excepción justificada (`NOSONAR`).

Consecuencias

- Las clases se prueban capturando la salida en memoria.
- La aplicación de consola mantiene exactamente el mismo formato de salida.

ADR 0005 — Estrategia de selección de la RCL como `enum`

Estado: aceptada

Contexto

La estrategia se elegía con `int selection = 3` dentro de `computeGreedySolution`, junto a `double alpha = 0.8`. Cambiarla requería editar el código, y el `switch` tenía dos ramas idénticas (`java:S1871`).

Decisión

Definir `TOPTWGRASP.SelectionStrategy { RANDOM, FUZZY_BEST, FUZZY_ALPHA_CUT }` con *setters* para la estrategia y para `alpha`. Los valores por defecto (`FUZZY_ALPHA_CUT`, 0,8) son los del original.

Consecuencias

- Las estrategias se pueden comparar sin recompilar y se prueban las tres.
- El comportamiento por defecto no cambia.

ADR 0006 — Documentación Docs-as-Code con Doxygen y MkDocs

Estado: aceptada

Contexto

La práctica exige documentar el código con Doxygen (con grafos de Graphviz) y publicar un portal técnico unificado. Había que decidir cómo unir la referencia generada con las guías escritas a mano.

Opciones

1. HTML de Doxygen con el tema *Doxygen Awesome* y las guías como páginas de Doxygen.
2. MkDocs Material con las guías en Markdown y la referencia integrada con **MkDoxy**.

Decisión

Opción 2. MkDocs Material da mejor navegación, búsqueda y diseño para las guías; MkDoxy convierte la salida XML de Doxygen en páginas del mismo portal. El HTML completo de Doxygen (con los grafos interactivos) se publica también en `/doxygen/` y el PDF se obtiene con LaTeX (`refman.pdf`) y con un *script* que imprime el portal.

Consecuencias

- Un único portal con guías, decisiones y referencia del código.
- La construcción requiere Doxygen, Graphviz y Python; se automatiza en GitHub Actions.
- El *script* de PDF usa Playwright (Chromium) en lugar de `mkdocs-with-pdf`, que no se mantiene desde 2021 y no funciona con las versiones actuales de WeasyPrint.

Referencia de la API

La referencia del código se genera **automáticamente** a partir de los comentarios Doxygen en cada construcción del portal; no se edita a mano.

Formato	Contenido	Enlace
Páginas del portal (MkDoxy)	Clases, miembros, jerarquía y paquetes	Lista de clases
HTML de Doxygen	Igual, más grafos interactivos de herencia, colaboración y llamadas, y el código fuente navegable	Abrir
PDF (LaTeX)	Manual de referencia completo	ExpositoTOP-referencia-doxygen.pdf
PDF del portal	Este portal completo	ExpositoTOP-documentacion.pdf

Clases principales

Clase	Resumen
`top.TOPTW`	Modelo de una instancia del TOPTW
`top.TOPTWReader`	Lector de instancias en formato Cordeau/Righini
`top.TOPTWSolution`	Solución representada mediante listas doblemente enlazadas
`top.TOPTWEvaluator`	Evaluador de la función objetivo
`top.TOPTWGRASP`	Metaheurística GRASP
`top.TOPTWRoute`	Descriptor ligero de una ruta
`top.MainTOPTW`	Punto de entrada de la aplicación
`es.ull.esit.utilities.ExpositoUtilities`	Formato de tablas, ficheros, matrices y grafos
`es.ull.esit.utilities.PowerSet`	Iterador sobre el conjunto potencia
`es.ull.esit.utilities.BellmanFord`	Algoritmo de Bellman-Ford
`es.ull.esit.utils.Pair`	Par inmutable

Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

- **namespace** es
  - **namespace** ull
    - **namespace** esit
      - **namespace** utilities
        - **class** BellmanFord *Algoritmo de Bellman-Ford sobre una matriz de distancias.*
        - **class** ExpositoUtilities *Funciones de utilidad: formato de texto tabular, ficheros, matrices y grafos.*
        - **class** PowerSet *Conjunto potencia (todos los subconjuntos) de un conjunto, recorrible con for-each.*
      - **namespace** utils
- **class** PowerSetIterator *Iterador sobre los subconjuntos, basado en un contador binario.*
- **namespace** top
  - **class** MainTOPTW *Punto de entrada del recomendador turístico RTVRPTW / **TOPTW** .*
    - **class** Options *Opciones de ejecución obtenidas de la línea de órdenes.*
  - **class** TOPTW *Modelo de una instancia del problema **TOPTW** (Team Orienteering Problem with Time Windows).*
  - **class** TOPTWEvaluator *Evaluador de la función objetivo del **TOPTW** .*
  - **class** TOPTWGRASP *Metaheurística GRASP (Greedy Randomized Adaptive Search Procedure) para el **TOPTW** .*
  - **class** TOPTWReader *Lector de instancias **TOPTW** en el formato de Cordeau / Righini.*
  - **class** TOPTWRoute *Descriptor ligero de una ruta: identificador y nodos extremos.*
  - **class** TOPTWSolution *Solución del problema **TOPTW** representada mediante listas doblemente enlazadas.*

Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- **class** **es::ull::esit::utilities::BellmanFord** *Algoritmo de Bellman-Ford sobre una matriz de distancias.*
- **class** **es::ull::esit::utilities::ExpositoUtilities** *Funciones de utilidad: formato de texto tabular, ficheros, matrices y grafos.*
- **class** **top::MainTOPTW** *Punto de entrada del recomendador turístico RTVRPTW / **TOPTW** .*
- **class** **top::MainTOPTW::Options** *Opciones de ejecución obtenidas de la línea de órdenes.*
- **class** **top::TOPTW** *Modelo de una instancia del problema **TOPTW** (Team Orienteering Problem with Time Windows).*
- **class** **top::TOPTWEvaluator** *Evaluador de la función objetivo del **TOPTW** .*
- **class** **top::TOPTWGRASP** *Metaheurística GRASP (Greedy Randomized Adaptive Search Procedure) para el **TOPTW** .*
- **class** **top::TOPTWReader** *Lector de instancias **TOPTW** en el formato de Cordeau / Righini.*
- **class** **top::TOPTWRoute** *Descriptor ligero de una ruta: identificador y nodos extremos.*
- **class** **top::TOPTWSolution** *Solución del problema **TOPTW** representada mediante listas doblemente enlazadas.*
- **class Iterable< Set< E > >**
  - **class** **es::ull::esit::utilities::PowerSet** *Conjunto potencia (todos los subconjuntos) de un conjunto, recorrible con for-each.*
- **class java::util::Iterator< Set< E > >**
  - **class** **es::ull::esit::utilities::PowerSet::PowerSetIterator** *Iterador sobre los subconjuntos, basado en un contador binario.*

Class Members

a

- **ALIGNMENT_LEFT** (es::ull::esit::utilities::ExpositoUtilities)
- **ALIGNMENT_RIGHT** (es::ull::esit::utilities::ExpositoUtilities)
- **arr** (es::ull::esit::utilities::PowerSet)
- **addNode** (top::TOPTW)
- **addNodeDepot** (top::TOPTW)
- **aleatorySelectionRCL** (top::TOPTWGRASP)
- **alpha** (top::TOPTWGRASP)
- **addRoute** (top::TOPTWSolution)
- **availableVehicles** (top::TOPTWSolution)

b

- **BellmanFord** (es::ull::esit::utilities::BellmanFord)
- **bset** (es::ull::esit::utilities::PowerSet::PowerSetIterator)

c

- **calculateEdges** (es::ull::esit::utilities::BellmanFord)
- **calculateDistanceMatrix** (top::TOPTW)
- **CANDIDATE_INFO_SIZE** (top::TOPTWGRASP)
- **COST** (top::TOPTWGRASP)
- **CUSTOMER** (top::TOPTWGRASP)
- **comprehensiveEvaluation** (top::TOPTWGRASP)
- **computeGreedySolution** (top::TOPTWGRASP)

d

- **distanceMatrix** (es::ull::esit::utilities::BellmanFord, top::TOPTW)
- **distances** (es::ull::esit::utilities::BellmanFord)

- **DEFAULT_COLUMN_WIDTH** (es::ull::esit::utilities::ExpositoUtilities)
- **DEFAULT_DIR** (top::MainTOPTW)
- **DEFAULT_INSTANCES** (top::MainTOPTW)
- **DEFAULT_ITERATIONS** (top::MainTOPTW)
- **dir** (top::MainTOPTW::Options)
- **dataIndex** (top::TOPTW)
- **depots** (top::TOPTW)
- **dueTime** (top::TOPTW)
- **DEFAULT_ALPHA** (top::TOPTWGRASP)
- **departureTime** (top::TOPTWGRASP)

e

- **edges1** (es::ull::esit::utilities::BellmanFord)
- **edges2** (es::ull::esit::utilities::BellmanFord)
- **ExpositoUtilities** (es::ull::esit::utilities::ExpositoUtilities)
- **evaluate** (top::TOPTWEvaluator)
- **equals** (top::TOPTWSolution)
- **evaluateFitness** (top::TOPTWSolution)

f

- **fuzzySelectionAlphaCutRCL** (top::TOPTWGRASP)
- **fuzzySelectionBestFDRCL** (top::TOPTWGRASP)

g

- **getDistances** (es::ull::esit::utilities::BellmanFord)
- **getValue** (es::ull::esit::utilities::BellmanFord)
- **getFormat** (es::ull::esit::utilities::ExpositoUtilities)
- **getFormatIntegers** (es::ull::esit::utilities::ExpositoUtilities)
- **getDepots** (top::TOPTW)
- **getDistance** (top::TOPTW, top::TOPTWSolution)

- **getDueTime** (top::TOPTW)
- **getMaxRoutes** (top::TOPTW)
- **getMaxTimePerRoute** (top::TOPTW)
- **getNodes** (top::TOPTW)
- **getPOIs** (top::TOPTW)
- **getReadyTime** (top::TOPTW)
- **getScore** (top::TOPTW)
- **getServiceTime** (top::TOPTW)
- **getTime** (top::TOPTW)
- **getTotalDistance** (top::TOPTW)
- **getVehicles** (top::TOPTW)
- **getX** (top::TOPTW)
- **getY** (top::TOPTW)
- **getAlpha** (top::TOPTWGRASP)
- **getLastAverageFitness** (top::TOPTWGRASP)
- **getLastBestFitness** (top::TOPTWGRASP)
- **getMaxScore** (top::TOPTWGRASP)
- **getSolution** (top::TOPTWGRASP)
- **getSolutionTime** (top::TOPTWGRASP)
- **getStrategy** (top::TOPTWGRASP)
- **grasp** (top::TOPTWGRASP)
- **getId** (top::TOPTWRoute)
- **getPredecessor** (top::TOPTWRoute, top::TOPTWSolution)
- **getSuccessor** (top::TOPTWRoute, top::TOPTWSolution)
- **getAvailableVehicles** (top::TOPTWSolution)
- **getCreatedRoutes** (top::TOPTWSolution)
- **getIndexRoute** (top::TOPTWSolution)
- **getInfoSolution** (top::TOPTWSolution)
- **getObjectiveFunctionValue** (top::TOPTWSolution)
- **getPositionInRoute** (top::TOPTWSolution)
- **getPredecessors** (top::TOPTWSolution)
- **getProblem** (top::TOPTWSolution)

- **getSuccessors** (top::TOPTWSolution)
- **getWaitingTime** (top::TOPTWSolution)

h

- **hasArc** (es::ull::esit::utilities::ExpositoUtilities)
- **hasNext** (es::ull::esit::utilities::PowerSet::PowerSetIterator)
- **hashCode** (top::TOPTWSolution)

i

- **INFINITY** (es::ull::esit::utilities::BellmanFord)
- **isAcyclic** (es::ull::esit::utilities::ExpositoUtilities)
- **isDouble** (es::ull::esit::utilities::ExpositoUtilities)
- **isInteger** (es::ull::esit::utilities::ExpositoUtilities)
- **iterator** (es::ull::esit::utilities::PowerSet)
- **instances** (top::MainTOPTW::Options)
- **iterations** (top::MainTOPTW::Options)
- **isDepot** (top::TOPTW, top::TOPTWSolution)
- **insertionCost** (top::TOPTWGRASP)
- **id** (top::TOPTWRoute)
- **initSolution** (top::TOPTWSolution)

l

- **LOGGER** (top::MainTOPTW)
- **lastAverageFitness** (top::TOPTWGRASP)
- **lastBestFitness** (top::TOPTWGRASP)

m

- **multiplyMatrices** (es::ull::esit::utilities::ExpositoUtilities)
- **MainTOPTW** (top::MainTOPTW)
- **main** (top::MainTOPTW)

- **maxRoutes** (top::TOPTW)
- **maxTimePerRoute** (top::TOPTW)
- **membership** (top::TOPTWGRASP)

n

- **nodes** (es::ull::esit::utilities::BellmanFord, top::TOPTW)
- **next** (es::ull::esit::utilities::PowerSet::PowerSetIterator)
- **NO_EVALUATED** (top::TOPTWEvaluator)
- **newDepartureTimes** (top::TOPTWGRASP)
- **nextFields** (top::TOPTWReader)
- **NO_INITIALIZED** (top::TOPTWSolution)

o

- **optionValue** (top::MainTOPTW)
- **out** (top::TOPTWGRASP)
- **objectiveFunctionValue** (top::TOPTWSolution)

p

- **path** (es::ull::esit::utilities::BellmanFord)
- **printFile** (es::ull::esit::utilities::ExpositoUtilities)
- **PowerSet** (es::ull::esit::utilities::PowerSet)
- **parseArgs** (top::MainTOPTW)
- **PREDECESSOR** (top::TOPTWGRASP)
- **print** (top::TOPTWGRASP)
- **predecessor** (top::TOPTWRoute)
- **positionInRoute** (top::TOPTWSolution)
- **predecessors** (top::TOPTWSolution)
- **printSolution** (top::TOPTWSolution)
- **problem** (top::TOPTWSolution)

r

- **RCL_SIZES** (top::MainTOPTW)
- **run** (top::MainTOPTW)
- **readyTime** (top::TOPTW)
- **ROUTE** (top::TOPTWGRASP)
- **random** (top::TOPTWGRASP)
- **readProblem** (top::TOPTWReader)
- **routes** (top::TOPTWSolution)

S

- **solve** (es::ull::esit::utilities::BellmanFord)
- **simplifyString** (es::ull::esit::utilities::ExpositoUtilities)
- **seed** (top::MainTOPTW::Options)
- **score** (top::TOPTW)
- **serviceTime** (top::TOPTW)
- **setDueTime** (top::TOPTW)
- **setMaxRoutes** (top::TOPTW)
- **setMaxTimePerRoute** (top::TOPTW)
- **setNodes** (top::TOPTW)
- **setReadyTime** (top::TOPTW)
- **setScore** (top::TOPTW)
- **setServiceTime** (top::TOPTW)
- **setX** (top::TOPTW)
- **setY** (top::TOPTW)
- **SCORE** (top::TOPTWGRASP)
- **select** (top::TOPTWGRASP)
- **setAlpha** (top::TOPTWGRASP)
- **setOutput** (top::TOPTWGRASP)
- **setSolution** (top::TOPTWGRASP)
- **setSolutionTime** (top::TOPTWGRASP)
- **setStrategy** (top::TOPTWGRASP)

- **solution** (top::TOPTWGRASP)
- **solutionTime** (top::TOPTWGRASP)
- **sortedCandidates** (top::TOPTWGRASP)
- **strategy** (top::TOPTWGRASP)
- **SelectionStrategy** (top::TOPTWGRASP)
- **SCORE_FIELD** (top::TOPTWReader)
- **SERVICE_FIELD** (top::TOPTWReader)
- **setId** (top::TOPTWRoute)
- **setPredecessor** (top::TOPTWRoute, top::TOPTWSolution)
- **setSuccessor** (top::TOPTWRoute, top::TOPTWSolution)
- **successor** (top::TOPTWRoute)
- **setAvailableVehicles** (top::TOPTWSolution)
- **setObjectiveFunctionValue** (top::TOPTWSolution)
- **setPositionInRoute** (top::TOPTWSolution)
- **setWaitingTime** (top::TOPTWSolution)
- **successors** (top::TOPTWSolution)

t

- **thereIsPath** (es::ull::esit::utilities::ExpositoUtilities)
- **TOPTW** (top::TOPTW)
- **toString** (top::TOPTW)
- **TOPTWGRASP** (top::TOPTWGRASP)
- **TOPTWReader** (top::TOPTWReader)
- **TOPTWRoute** (top::TOPTWRoute)
- **TOPTWSolution** (top::TOPTWSolution)

u

- **updateSolution** (top::TOPTWGRASP)

V

- **value** (es::ull::esit::utilities::BellmanFord)
- **vehicles** (top::TOPTW)

W

- **writeTextToFile** (es::ull::esit::utilities::ExpositoUtilities)
- **waitingTime** (top::TOPTWSolution)

X

- **x** (top::TOPTW)
- **X_FIELD** (top::TOPTWReader)

y

- **y** (top::TOPTW)
- **Y_FIELD** (top::TOPTWReader)

Namespace List

Here is a list of all namespaces with brief descriptions:

- **namespace** es
  - **namespace** ull
    - **namespace** esit
      - **namespace** utilities
      - **namespace** utils
- **namespace** top

File List

Here is a list of all files with brief descriptions:

- dir src
  - dir main
    - dir java
      - dir es
        - dir ull
          - dir esit
            - dir utilities
              - file BellmanFord.java
              - file ExpositoUtilities.java
              - file PowerSet.java
            - dir utils
              - file Pair.java
      - dir top
        - file MainTOPTW.java
        - file TOPTW.java
        - file TOPTWEvaluator.java
        - file TOPTWGRASP.java
        - file TOPTWReader.java
        - file TOPTWRoute.java
        - file TOPTWSolution.java

Registro de cambios

Formato basado en Keep a Changelog.

[1.0.0] — Práctica 3

Añadido

- Proyecto Maven (`pom.xml`) con enforcer, compiler, surefire, JaCoCo, jar, source, javadoc, Checkstyle, PMD, SpotBugs, exec y SonarQube; perfil `docs`.
- 43 tests JUnit y una instancia de prueba mínima.
- Opciones de línea de órdenes `--dir`, `--iterations` y `--seed`.
- `TOPTWGRASP.SelectionStrategy` y *setters* de estrategia, `alpha` y salida.
- `TOPTWEvaluator.evaluate` implementado.
- Documentación Doxygen en todas las clases, `Doxyfile` con Graphviz, portal MkDocs Material con MkDoxy, ADR y PDF.
- GitHub Actions: construcción, tests, análisis SonarQube y publicación en GitHub Pages.

Corregido

- Lectura de la ventana temporal en instancias con listas de longitud variable (`pr*`).
- `TOPTWReader` terminaba el proceso con código 0 ante errores.
- `NullPointerException` en `ExpositoUtilities.printFile` si el fichero no existe.
- Recurso sin cerrar en `ExpositoUtilities.writeTextToFile`.
- `PowerSet.next()` no lanzaba `NoSuchElementException`.
- `TOPTWSolution.equals` no sobrescribía `Object.equals`; faltaba `hashCode`.
- Columna `SERVICE TIME` y celdas obsoletas en el informe de solución.

Cambiado

- `mainTOPTW` → `MainTOPTW`; `GRASP()` → `grasp()` (devuelve la mejor puntuación).
- `TOPTWRoute.getPredeccesor/getSuccesor` → `getPredecessor/getSuccessor`.
- `ExpositoUtilities.getFormat(ArrayList<Integer>)` → `getFormatIntegers(List<Integer>)`.
- `TOPTW.getDistance(ArrayList<Integer>[])` → `getTotalDistance(List<List<Integer>>)`.

- Reducción de la complejidad cognitiva de la construcción voraz sin cambiar los resultados.

Eliminado

- `build/`, `dist/`, `nbproject/`, `build.xml`, ficheros de IDE y `ExpositoUtilities.java~`.
- Código comentado y miembros sin uso.